4

Data science

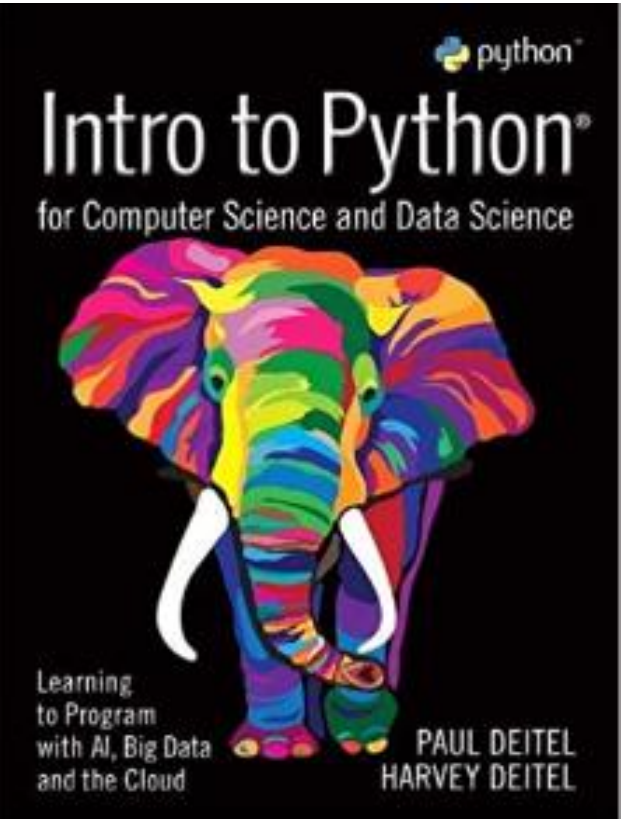
© 2008 Pearson Education, Inc. All rights reserved.

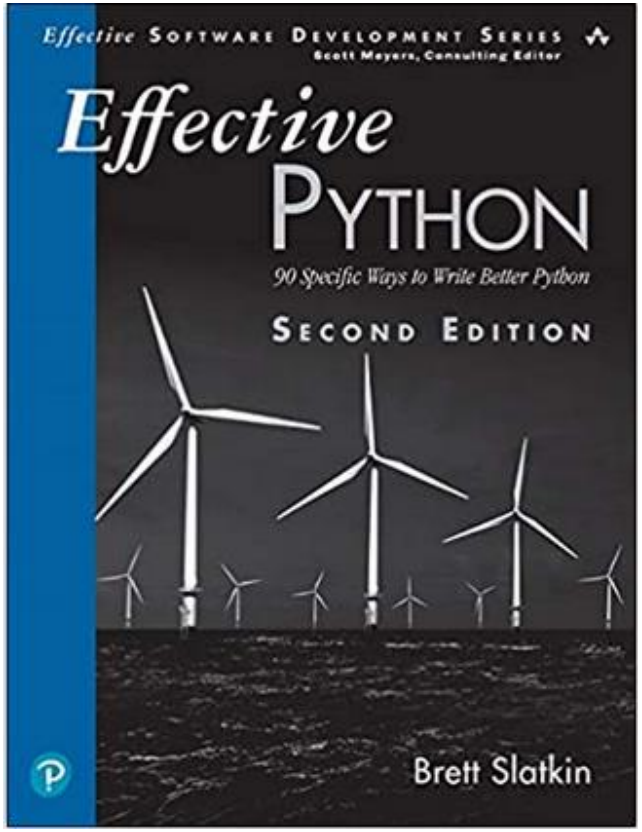
Inhoud

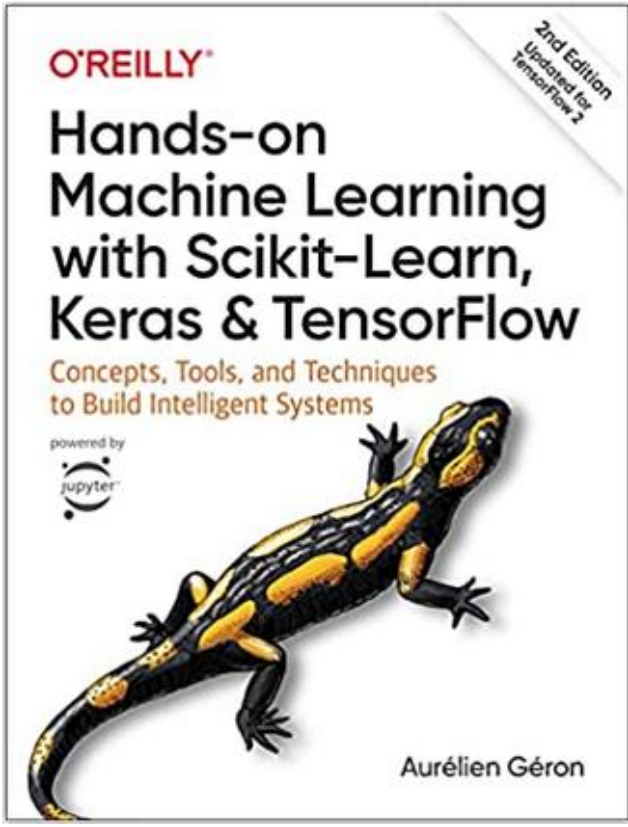
1. Inleiding
2. Programmeren onder Linux
3. Robuust ontwerp in Python
4. Data Science & ML
    4.1. Data science
    4.2. Pandas Series
    4.3. Pandas DataFrame

© 2008 Pearson Education, Inc. All rights reserved.

Referenties







© 2008 Pearson Education, Inc. All rights reserved.

4.1 Data Science

- Data krijgt meer prominente rol dan software zelf
- Heel wat internet bedrijven vergaren data: Google, Facebook, …
- Meten is weten: Door afnemende prijs van sensoren, hun gebruiksgemak en hun connectivity (wireless, eenvoudige netwerken, …) rusten we zoveel mogelijk producten en omgevingen uit met sensoren die data genereren(Internet of things)
- Men verwacht dat tegen 2025 er 50 miljard toestellen op het interet zullen aangesloten zijn
- Volgens IBM wordt er elke dag wereldwijd 2.5 exabytes (10^{18} bytes) aan nieuwe data gegenereerd
- Elke person op aarde produceert gemiddeld 1.7MB data per seconde: per dag genereren we evenveel data als 5.5M uur aan 4K video (600jaar 4K video)
- 90% van de huidige data in de wereld is de laatste 2 jaar gecreëerd
- De globale hoeveelheid data wordt tegen 2025 geschat op 175 zetabytes (10^{21} bytes)

© 2008 Pearson Education, Inc. All rights reserved.

4.1 Data Science

- Voor overzichten: https://www.internetlivestats.com/
- Ook computing power neemt drastisch toe:
- FLOPS: Floating Point Operations per Second
- Snelste supercomputers in Jaren ‘90: gigaFLOPS (10^9 FLOPS) – teraFLOPS (10^{12} FLOPS)
- Huidige snelse supercomputer (Oak Ridge National Laboratory): 122.3 petaFLOPS
- Ook publiek toegankelijke computing power: via cloud en distributed computing: pay for what you need

© 2008 Pearson Education, Inc. All rights reserved.

4.1 Data Science

- Big Data en Data Science is een goed ontwikkelde academische en bedrijfstak
- 4 Belangrijke factoren (4V's):
  - Volume: groeit exponentieel met de tijd
  - Velocity: de snelheid waarmee data wordt geproduceerd, de snelheid waarmee deze data verwerkt wordt en de snelheid waarmee data verandert
  - Variety: de verscheidenheid aan data: teksten, beelden, audio, financieel, urban, mobiliteit, domotica, sensors, …
  - Veracity: de validiteit, de juistheid, echtheid, relevantie, …
- Data science: extraheer kennis uit data door het gemengd gebruik van statistiek, data analyse en computerwetenschap
- Maak gebruik van: clustering en regressie, modellering en simulatie, compressie, patroonherkenning, machine learning, AI, visualisatie, parallelisatie, ….
- Economische relevantie:
  - Toenemende winsten: snel reageren op veranderingen, modelleren en voorspellen, automatisatie, ….
  - Betere kennis van menselijk gedrag: klantenprofiel, electorale profielen, …
  - Mobiliteit en veiligheid
  - Medische vooruitgang
  - Sportwetenschappen

© 2008 Pearson Education, Inc. All rights reserved.

4.1 Public Data

- Waar haal je datasets vandaan?
- Je eigen simulaties of experimentele data
- Scikit-learn package heeft eigen datsets: https://scikit-learn.org/stable/datasets.html
- Public data repositories:
  - OpenML datset: https://openml.org/

    bevat 21.541 datasets in handige zoektabel

    scikit-learn heeft modules om deze data te downloaden : fetch_openml
  - Kaggle datsets: https://www.kaggle.com/

    bevat 50.000 publieke datsets in verschillende domeinen

    bvb. De Titanic dataset
  - UC Irvine Machine Learning Repository: https://archive.ics.uci.edu/ml/index.php

    Bevat momenteel 622 datasets, getabueerd volgens grootte, attribute etc.: https://archive.ics.uci.edu/ml/datasets.php

    bvb de Iris data set: https://archive.ics.uci.edu/ml/datasets/Iris
  - Amazon AWS datsets: https://registry.opendata.aws/

    bevat 320 datasets

    Je moet registreren: https://aws.amazon.com/

    Je kan amazon computing resources gebruiken op de ‘cloud’
  - Voor uitgebreid overzicht, zie: https://opendatamonitor.eu

© 2008 Pearson Education, Inc. All rights reserved.

4.1 Downloading data in Python

- Je kan data downloaden via website en opslaan in je project
- Je kan ook een Python functie schrijven die de data download: handig indien data regelmatig verandert of aangevuld wordt
- Gebruik de Python standard libraries: os, urllib, tarfile, …

```
import os
import urllib.request
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import linear_model

DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml2/master/"
DATA_URL = DOWNLOAD_ROOT + "datasets/lifesat/"
DATA_PATH = os.path.join("datasets", "lifesat", "")
def fetch_data(data_url=DATA_URL, data_path=DATA_PATH):
    os.makedirs(data_path, exist_ok=True)
    for filename in ("oecd_bli_2015.csv", "gdp_per_capita.csv"):
        print("Downloading", filename)
        urllib.request.urlretrieve(data_url + filename, data_path + filename)


fetch_data()
```

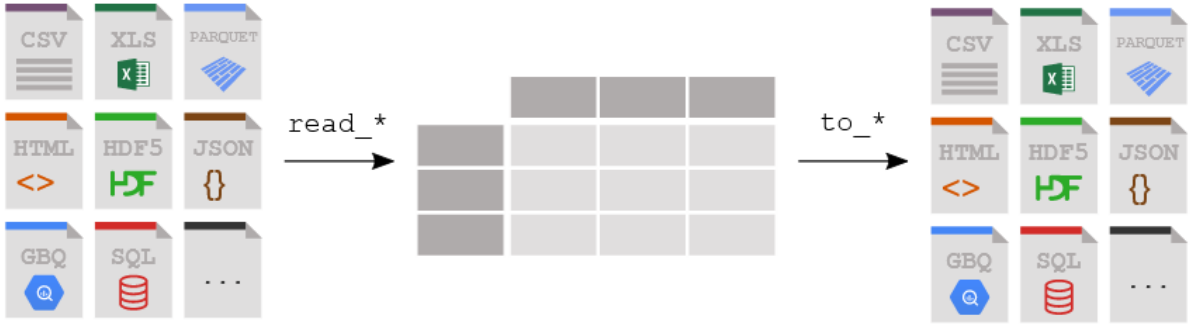
Maakt lokale directories in je project folder

Download bestanden naar de lokale folders

© 2008 Pearson Education, Inc. All rights reserved.

4.2 Pandas

- **Data is beschikbaar in tal van formaten**



- **Data kan opgeslagen worden in tal van Python data structuren: lists, tuples, dictionaries, sets, maps, NumPy arrays, …**
- **Large Data (<100GB) vs Big Data (>100GB)**
- **Pandas:**
  - **Open source library sinds 2009**
  - **Performant en flexibele manipulatie van logisch gestructureerde data**
  - **Is bovenop NumPy geimplementeerd en bijgevolg erg compatible**
  - **Performant voor datasets van 100MB-1GB**
  - **Stilaan de standaard in vele academische en commerciele toepassingen**
- **Installatie via:** `pip install pandas`
- **Check versie via:** `pip show pandas`

© 2008 Pearson Education, Inc. All rights reserved.

4.2 Pandas

- **Voor complete info zie: https://pandas.pydata.org/docs/index.html**
- **Twee belangrijkste klassen**
  - **Series: enhanced 1D array data structuur**
  - **DataFrame: enhanced 2D array,**
- **Series:**
  - **lineaire data structuur die elementen met gemengde types kan bevatten**
  - **Custom indexering met non-integer indices mogelijk**
  - **Mogelijkheid tot behandelen van ontbrekende data**
- **DataFrame:**
  - **2d, size-mutable, heterogene data structuur**
  - **elke kolom is een Series**
  - **Beide assen zijn gelabeld (rijen en kolommen)**

© 2008 Pearson Education, Inc. All rights reserved.

4.2 Series

- Pandas Series zijn 1d gelabelde arrays die eender welk data type kunnen bevatten
- Als het ware dict met een vaste lengte
- De axis labels worden de index genoemd
- Syntax s=pd.Series(<data>, index=<index>)
- De <data> kan zijn:
  - Python dict
  - NumPy ndarray (in dit geval moet index zelfde lengte hebben)
  - Scalar
- Panda data structuren zijn in staat om ‘ontbrekende data’ te behandelen: NaN is standard data type voor missing data
- Series zijn erg gelijkaardig aan ndarrays en kunnen als argument aan de meeste NumPy functies worden gegeven
- Indien je enkel de datavelden nodig hebt, gebruik dan attribute **`array`**
- Indien je de ruwe ndarray nodig hebt, gebruik **`to_numpy()`** method

© 2008 Pearson Education, Inc. All rights reserved.

4.2.1 Series voorbeeld

```python
import pandas as pd
import numpy as np

## Working with Pandas Series
############################
# Simplest use of pandas Series
grades = pd.Series([87, 100, 94])
print(grades)
print(grades.describe())
# access to individual data values with zero-based integer index
print(grades[1])
# make a Series with custom indices
grades = pd.Series([87, 100, 94], index=['Wally', 'Bruce', 'Mary Jane'])
# access to individual data values with custom index
print(grades['Mary Jane'])
# access to all values
print(grades.values)
# access to all custom indices
print(grades.index)
# obtain statistical attributes
print(grades.count())
print(grades.mean())
print(grades.min())
```

Toegang via integer based index

Toegang via custom index

Toegang tot alle waarden

Toegang tot alle indeces

Statistische eigenschappen

© 2008 Pearson Education, Inc. All rights reserved.

4.2.1 Series voorbeeld

```
# make a Series from a dict
d = {"a": 0.0, "b": 1.0, "c": 2.0}
s = pd.Series(d)
print(s)
# make a Series by reshuffling data fields from a dict and add additional field index d
v = pd.Series(d, index=["b", "c", "d", "a"])
print(v)
mypar=v.array
print(mypar)
myndar=v.to_numpy()
print(myndar)
# apply filters to the data values
print(id(v))
print(v[v<2.0])
# remove the NaN values (note that filters do not change
# the original Series but return a new one
print(v.dropna())
print(v)
print(id(v))
# Doing the same with a lambda expression
print(v.where(lambda x : x!=1).dropna())
```

When you add an index without associated data, the data will be set to NaN

Convert to raw array data

Convert to NumPy array

Boolean indexing to filter data

Remove NaN values

Filtering using lambda expressions

© 2008 Pearson Education, Inc. All rights reserved.

4.2.1 Series voorbeeld

```
# making a series of strings
hardware = pd.Series(['hammer', 'saw', 'wrench'])
print(hardware)
# now use a string method to filter the Series
# notice the difference in output between the two following
# statements
print(hardware.str.contains('a'))
# now actually transform the series
print(hardware[hardware.str.contains('a')])
# if you want to negate the selection use ~
print(hardware[~hardware.str.contains('a')])
```

Using a string method for filtering. The output will be a Series of Booleans

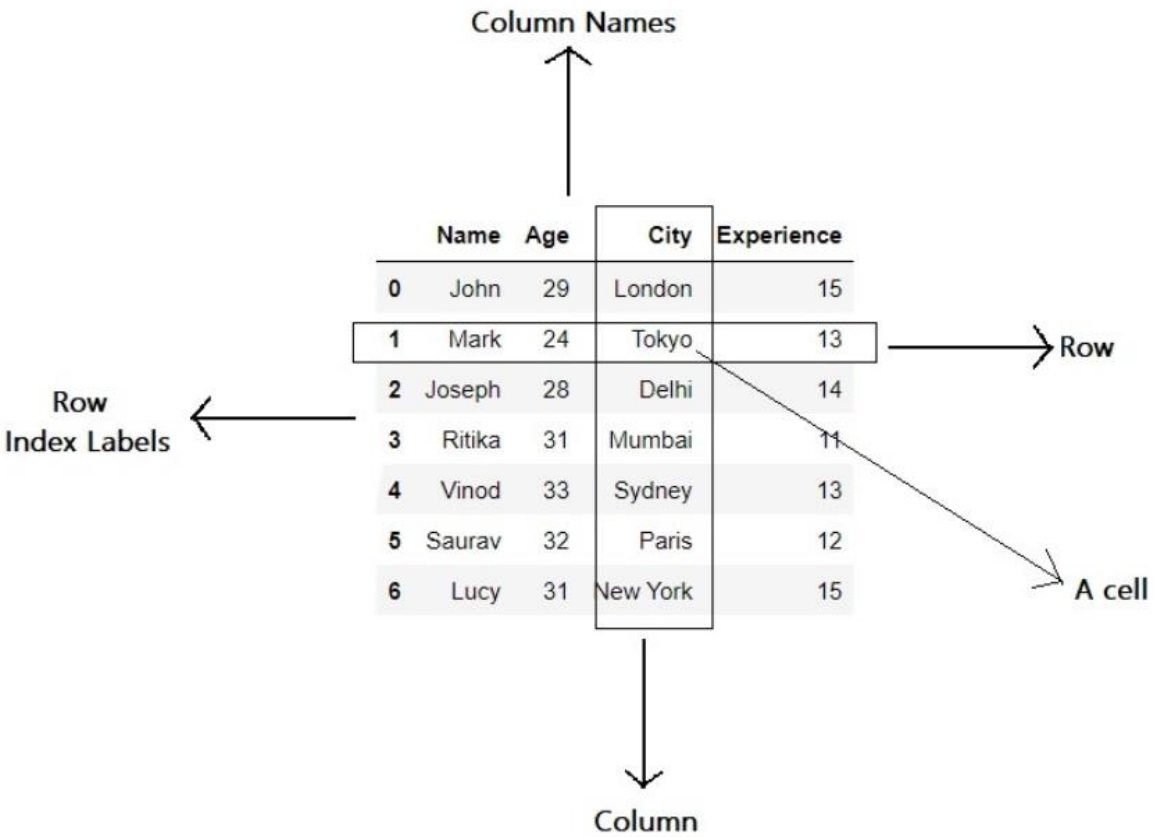
Notice the advanced use of boolean indexing. Now we have a filtered Series

Negation (complement) of a filter is obtained by using ~

© 2008 Pearson Education, Inc. All rights reserved.

4.3 DataFrame

- 2D gelabelde data structuur met kolommen van mogelijk verschillende data types
- Leunt sterk aan bij spreadsheet, database
- Kan gezien worden als een dict van Series



	Name	Age	City	Experience
0	John	29	London	15
1	Mark	24	Tokyo	13
2	Joseph	28	Delhi	14
3	Ritika	31	Mumbai	11
4	Vinod	33	Sydney	13
5	Saurav	32	Paris	12
6	Lucy	31	New York	15

- DataFrame kan als input krijgen:
  - Dict of 1D ndarrays, lists, dicts, or Series
  - 2-D numpy.ndarray
  - Structured or record ndarray
  - Series
  - Een ander DataFrame

© 2008 Pearson Education, Inc. All rights reserved.

4.3 DataFrame

- Creatie van een DataFrame: zie ook https://pandas.pydata.org/docs/reference/frame.html

  pandas.DataFrame(*data=None, index=None, columns=None, dtype=None, copy=None*)

- data: ndarray (structured or homogeneous), Iterable, dict, or DataFrame
  Dict can contain Series, arrays, constants, dataclass or list-like objects. If data is a dict, column order follows insertion-order. If a dict contains Series which have an index defined, it is aligned by its index.
- index: Index or array-like
  Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided.
- columns: Index or array-like
  Column labels to use for resulting frame when data does not have them, defaulting to RangeIndex(0, 1, 2, …, n). If data contains column labels, will perform column selection instead.
- dtype: dtype, default None
  Data type to force. Only a single dtype is allowed. If None, infer
- copy: bool or None, default None
  Copy data from inputs. For dict data, the default of None behaves like copy=True. For DataFrame or 2d ndarray input, the default of None behaves like copy=False.

© 2008 Pearson Education, Inc. All rights reserved.

4.3 DataFrame

- Panda functies die informatie verschafen:
  - **pandas.set_option(pat, value) = <pandas._config.config.CallableDynamicDoc object>**
    bvb. pd.set_option('display.max_columns', 100)
  - **DataFrame.head(*n=5*)**

    returns first n rows of the frame
  - **DataFrame.tail(*n=5*)**
    returns last n rows of the frame
  - **DataFrame.columns**

    Returns the list of column labels
  - **DataFrame.dtypes**

    Return the dtypes in the DataFrame
  - **DataFrame.count(axis=0, level=None, numeric_only=False)**
    Count non-NA cells for each column or row.
  - **Dataframe.len()**
    Is een Pyhton functie: geeft aantal rijen weer in een Frame
  - **DataFrame.info(verbose=None, buf=None, max_cols=None, memory_usage=None, show_counts=None, null_counts=None)**
    Print a concise summary of a DataFrame.
  - **DataFrame.value_counts(subset=None, normalize=False, sort=True, ascending=False, dropna=True)**
    Return a Series containing counts of unique rows in the DataFrame.

© 2008 Pearson Education, Inc. All rights reserved.

4.3 DataFrame

- Panda functies die opzoeken en of transformeren
  - **DataFrame.query(*expr, inplace=False, **kwargs*)**
    Query the columns of a DataFrame with a boolean expression
    expr: str
    The query string to evaluate.
    You can refer to variables in the environment by prefixing them with an '@' character like @a + b.
    You can refer to column names that are not valid Python variable names by surrounding them in backticks.
  - **DataFrame.drop(*labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise'*)**
    Remove rows or columns by specifying label names and corresponding axis, or by specifying directly index or column names.
  - **DataFrame.sample(n=None, frac=None, replace=False, weights=None, random_state=None, axis=None, ignore_index=False)**
    Return a random sample of items from an axis of object.
  - **DataFrame.select_dtypes(include=None, exclude=None)**
    Return a subset of the DataFrame's columns based on the column dtypes
  - **DataFrame.insert(loc, column, value, allow_duplicates=False)**
    Insert column into DataFrame at specified location.
  - **DataFrame.cumsum(axis=None, skipna=True, *args, **kwargs)**
    Return cumulative sum over a DataFrame or Series axis
  - **pandas.unique(values)**
    Return unique values based on a hash table (can be applied to any Pandas Series or DataFrame)
  - **DataFrame.isin(values)**
    Whether each element in the DataFrame is contained in values
  - **DataFrame.replace(to_replace=None, value=NoDefault.no_default, inplace=False, limit=None, regex=False, method=NoDefault.no_default)**
    Replace values given in to_replace with value
  - **DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=NoDefault.no_default, observed=False, dropna=True)**
    Group DataFrame using a mapper or by a Series of columns

© 2008 Pearson Education, Inc. All rights reserved.

4.3 DataFrame

- Data I/O: https://pandas.pydata.org/docs/reference/io.html
  - **pandas.read_csv(filepath_or_buffer, sep=NoDefault.no_default, delimiter=None, header='infer', names=NoDefault.no_default, index_col=None, usecols=None, squeeze=None, prefix=NoDefault.no_default, mangle_dupe_cols=True, dtype=None, engine=None, converters=None, true_values=None, false_values=None, skipinitialspace=False, skiprows=None, skipfooter=0, nrows=None, na_values=None, keep_default_na=True, na_filter=True, verbose=False, skip_blank_lines=True, parse_dates=None, infer_datetime_format=False, keep_date_col=False, date_parser=None, dayfirst=False, cache_dates=True, iterator=False, chunksize=None, compression='infer', thousands=None, decimal='.', lineterminator=None, quotechar='"', quoting=0, doublequote=True, escapechar=None, comment=None, encoding=None, encoding_errors='strict', dialect=None, error_bad_lines=None, warn_bad_lines=None, on_bad_lines=None, delim_whitespace=False, low_memory=True, memory_map=False, float_precision=None, storage_options=None)**

    voor alle parameters zie:
    https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html#pandas.read_csv
  - Volgende read functies bestaan ook:
    - read_excel(io[, sheet_name, header, names, ...])
    - read_json([path_or_buf, orient, typ, dtype, ...])
    - read_pickle(filepath_or_buffer[, ...])
    - read_html(io[, match, flavor, header, ...])
    - read_xml(path_or_buffer[, xpath, ...])
  - En er bestaan ook write functies natuurlijk:
    - DataFrame.to_pickle(path[, compression, ...])
    - DataFrame.to_csv([path_or_buf, sep, na_rep, ...])
    - DataFrame.to_excel(excel_writer[, ...])
    - DataFrame.to_json([path_or_buf, orient, ...])
    - DataFrame.to_html([buf, columns, col_space, ...])
    - DataFrame.to_latex([buf, columns, ...]) -→ Dit is een leuke

© 2008 Pearson Education, Inc. All rights reserved.

4.3.1 Data Frame voorbeeld

```
# make a data frame out of a dict of Series
d = {
    "one": pd.Series([1.0, 2.0, 3.0], index=["a", "b", "c"]),
    "two": pd.Series([1.0, 3.0, 4.0, 2.0], index=["a", "b", "c", "d"]),
    }
df = pd.DataFrame(d)
print(df)
# accessing data frames columns
print(df.one)
print(df.two)
# selecting rows with the custom index
print(df.loc['a'])
print(df.loc['b'])
print(df.loc['c'])
print(df.loc['d'])
# accessing specific data field
print(df.at['c','two'])
# doing the same with the zero-based integer index
print(df.iloc[0])
print(df.iloc[1])
print(df.iloc[2])
print(df.iloc[3])
print(df.iat[2,1])
# selecting columns with the custom index
print(df['one'])
print(df['two'])
```

Columns can be accessed as an attribute via their name

Rows can be accessed via their custom index using loc()

Individual value access via custom row and colum indices using at()

Similar results obtained with the integer index using iloc() and iat()

© 2008 Pearson Education, Inc. All rights reserved.

4.3.1 Data Frame voorbeeld

```
# the same can be obtained with a filter
print('using a filter')
print(df.filter(['two']))
# selecting a subset of the DataFrame
print(df.loc['a':'c', ['one']])
# applying a filter
print(df.dropna())
# take note of the output generated by this filter
print(df[df < 3.0])
# more boolean indexing on columns
print('filtering only column two')
print(df['two'] < 3.0)
# applying the filter
print(df[df['two'] < 3.0])
# changing the custom indices (the row indices)
# make sure that you use the correct number of indices
# otherwise you get a ValueError exception
df.index = ['alpha', 'beta', 'gamma', 'delta']
print(df)
# change the column names
print(df.rename(columns={'one': 'col1', 'two': 'col2'}))
# descriptive statistics
print(df.describe())
```

Using the filter() method to select a column

Selecting a subset with the loc() method

Removing NaN values

Using boolean indexing to filter on all values

Using boolean indexing to filter on values in specific column. Note that the output is a DataFrame with Booleans

This statement actually produces a filtered copy of a DataFrame

Changing the row indices

Changing the column names

© 2008 Pearson Education, Inc. All rights reserved.

4.3.1 Data Frame voorbeeld

```
# Transposing a data frame
print(df.T)
# get the stats per row
print(df.T.describe())
# sorting a data frame by index
print(df.sort_index(ascending=False))
# sorting the data frame by column label
print(df.sort_index(axis=1, ascending=False))
# sorting the data rows frama by value in spcific column
print(df.sort_values(by='two', ascending=False))
# sorting the data columns by valaue in specific row
print(df.sort_values(by='beta', axis=1, ascending=False))
# note that the sort method returns a sorted copy
# byt does not replace the original
print(df)
# if you want to overwrite the original, use inplace = true
print(df.sort_values(by='beta', axis=1, ascending=False, inplace=True))
print(df)
```

Transposing the rows and columns

Sorting according to row index names

Sorting according to column label via axis=1

Sorting rows by values in specific column

Sorting columns by specific values in a row

To avoid making a copy, use inplace=True (works with all filters and sorts)

© 2008 Pearson Education, Inc. All rights reserved.

4.3.2 Gevorderde Data Frame manipulaties

```
df2 = pd.DataFrame(
    {
        "A": ["A4", "A5", "A6", "A7"],
        "B": ["B4", "B5", "B6", "B7"],
        "C": ["C4", "C5", "C6", "C7"],
        "D": ["D4", "D5", "D6", "D7"],
    },
    index=['row4', 'row5', 'row6', 'row7'],
)
df3 = pd.DataFrame(
    {
        "A": ["A8", "A9", "A10", "A11"],
        "B": ["B8", "B9", "B10", "B11"],
        "E": ["E8", "E9", "E10", "E11"],
    },
    index=['row8', 'row1', 'row9', 'row10'],
)
```

```
df1 = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    },
    index=['row0', 'row1', 'row2', 'row3'],
)

# now we make a list of these 3 DataFrames
# before we use concat to concatenate them
frames = [df1, df2, df3]
result = pd.concat(frames)
# printing out the result
print(result)
print(result.loc['row2'])
print(result.loc['row1'])
```

```
         A    B    C    D    E
row0    A0   B0   C0   D0  NaN
row1    A1   B1   C1   D1  NaN
row2    A2   B2   C2   D2  NaN
row3    A3   B3   C3   D3  NaN
row4    A4   B4   C4   D4  NaN
row5    A5   B5   C5   D5  NaN
row6    A6   B6   C6   D6  NaN
row7    A7   B7   C7   D7  NaN
row8    A8   B8  NaN  NaN   E8
row1    A9   B9  NaN  NaN   E9
row9   A10  B10  NaN  NaN  E10
row10  A11  B11  NaN  NaN  E11
A     A2
B     B2
C     C2
D     D2
E    NaN
Name: row2, dtype: object
       A   B    C    D    E
row1  A1  B1   C1   D1  NaN
row1  A9  B9  NaN  NaN   E9
```

Concat will place rows one after the other. If they are additional colums they will be added and filled with NaN values for the missing data

© 2008 Pearson Education, Inc. All rights reserved.

4.3.2 Gevorderde Data Frame manipulaties

```python
# now we make a list of these 3 DataFrames
# before we use concat to concatenate them
frames = [df1, df2, df3]
result = pd.concat(frames, keys=['df1', 'df2', 'df3'])
# if you only want to retain the common column labels
# include option join='inner'
result2 = pd.concat(frames, keys=['df1', 'df2', 'df3'], join='inner')
# you can do the same for the rows
result3 = pd.concat([df1,df3], join='inner', axis=1)
# you can also ignore the index on the concatenation axis
result4 = pd.concat(frames, keys=['df1', 'df2', 'df3'], ignore_index=True, sort=False)
# printing out the result
print(result)
print(result.loc[['df1']])
print(result.xs('df3', level=0, axis=0, drop_level=True).loc['row1'])
print(result2)
print(result3)
print(result4)
```

```
df1 row0  A0  B0  C0  D0  NaN
    row1  A1  B1  C1  D1  NaN
    row2  A2  B2  C2  D2  NaN
    row3  A3  B3  C3  D3  NaN
A     A9
B     B9
C    NaN
D    NaN
E     E9
Name: row1, dtype: object
              A    B
df1 row0     A0   B0
    row1     A1   B1
    row2     A2   B2
    row3     A3   B3
df2 row4     A4   B4
    row5     A5   B5
    row6     A6   B6
    row7     A7   B7
df3 row8     A8   B8
    row1     A9   B9
    row9    A10  B10
    row10   A11  B11
        A   B   C   D   A   B   E
row1   A1  B1  C1  D1  A9  B9  E9
      A    B    C    D    E
0    A0   B0   C0   D0  NaN
1    A1   B1   C1   D1  NaN
2    A2   B2   C2   D2  NaN
```

© 2008 Pearson Education, Inc. All rights reserved.

4.3.2 Gevorderde Data Frame manipulaties

```
left = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
    }
)
right = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    }
)

result5 = pd.merge(left, right, on="key")
print(result5)
```

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	C2	D2
3	K3	A3	B3	C3	D3

```
left = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"]
    }, index = ["K0", "K1", "K2", "K3"],
)
right = pd.DataFrame(
    {
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"]
    }, index = ["K0", "K2", "K3", "K4"]
)
result6 = left.join(right, how ='inner')
print(result6)
```

	A	B	C	D
K0	A0	B0	C0	D0
K2	A2	B2	C1	D1
K3	A3	B3	C2	D2

© 2008 Pearson Education, Inc. All rights reserved.

4.3.2 Gevorderde Data Frame manipulaties

pivot

```
df
```

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

```
df.pivot(index='foo',
         columns='bar',
         values='baz')
```

bar	A	B	C
foo			
one	1	2	3
two	4	5	6

© 2008 Pearson Education, Inc. All rights reserved.

4.3.2 Gevorderde Data Frame manipulaties

```python
def unpivot(frame):

    N, K = frame.shape

    data = {

        "value": frame.to_numpy().ravel("F"),

        "variable": np.asarray(frame.columns).repeat(N),

        "date": np.tile(np.asarray(frame.index), K),

    }

    return pd.DataFrame(data, columns=["date", "variable", "value"])

tf = tm.makeTimeDataFrame(3)
print(tf)
df = unpivot(tf)
print(df)

print(df.pivot(index='date', columns='variable', values='value'))
```

```
                   A         B         C         D
2000-01-03 -1.090079  1.369043  0.542255  1.177432
2000-01-04  0.271854  0.045744  1.336280 -0.568472
2000-01-05  1.643401  0.065021  0.362525  0.434198
         date variable     value
0  2000-01-03        A -1.090079
1  2000-01-04        A  0.271854
2  2000-01-05        A  1.643401
3  2000-01-03        B  1.369043
4  2000-01-04        B  0.045744
5  2000-01-05        B  0.065021
6  2000-01-03        C  0.542255
7  2000-01-04        C  1.336280
8  2000-01-05        C  0.362525
9  2000-01-03        D  1.177432
10 2000-01-04        D -0.568472
11 2000-01-05        D  0.434198
variable           A         B         C         D
date
2000-01-03 -1.090079  1.369043  0.542255  1.177432
2000-01-04  0.271854  0.045744  1.336280 -0.568472
2000-01-05  1.643401  0.065021  0.362525  0.434198
```

© 2008 Pearson Education, Inc. All rights reserved.

Te Onthouden

1. Waar vind je data sets
2. Downloaden van data in Python
3. Pandas Series en DataFrame

© 2008 Pearson Education, Inc. All rights reserved.